The Paradigm Shift: From Model-Centric Scaling to Data-Centric Pipelines in Multimodal Video Contextualization

The prevailing trajectory in multimodal artificial intelligence relies heavily on the philosophy of indiscriminate parameter scaling to resolve complex reasoning deficits. However, the ingestion of continuous video data introduces a severe mathematical and hardware threshold. Because standard Transformer attention mechanisms scale quadratically, processing uncompressed, high-resolution video frames triggers an exponential explosion of visual tokens. This phenomenon exhausts GPU Virtual Random Access Memory (VRAM), induces catastrophic inference latency, and renders edge deployment mathematically impossible for massive Large Language Models (LLMs).[1] The industry's reliance on model-centric scaling to overcome this contextual bottleneck is computationally unsustainable in constrained hardware environments.

The core hypothesis under evaluation posits that the primary barrier in long-form video contextualization is not a deficit in parametric reasoning capacity, but rather the inefficiency of noisy, uncurated data-ingestion pipelines. By decoupling visual perception from textual reasoning—utilizing a data-centric, highly parallelized frame-sampling and spatial-temporal compression architecture—a Small Language Model (SLM) can demonstrate competitive, if not superior, temporal reasoning capabilities against a substantially larger LLM under the exact same computational budget.[1]

To empirically validate this hypothesis within a strict two-day execution window on Kaggle's dual NVIDIA T4 GPU infrastructure, the original theoretical framework must be aggressively adapted. The evaluation suite is restricted exclusively to the MVBench benchmark, which natively measures dynamic temporal understanding over static spatial memorization.[2] To guarantee perfect execution within Kaggle's thirty-hour weekly compute limit and twelve-hour session maximums, the LLM baseline is adjusted from an unfeasible 72B parameter scale to a 14B parameter scale utilizing 4-bit Activation-aware Weight Quantization (AWQ).[3] This strategic adjustment ensures the model fits perfectly within the 32GB cumulative VRAM of the dual T4 environment while preserving the absolute integrity of the SLM versus LLM comparative control.[1]

The Contextual Bottleneck and Hardware Realities

Standard Vision-Language Models (VLMs) process image patches at a virtually one-to-one ratio, often generating hundreds of tokens per single image frame. When deployed on video sequences, this payload is multiplied across the temporal axis, rapidly overwhelming the language model's context window. The computational cost incurred by these visual tokens heavily outweighs the text prompts, driving up inference latency and memory requirements.[1] Deploying such models in edge environments or constrained cloud notebooks requires a

profound understanding of the underlying hardware limitations.

The Kaggle environment provisions two NVIDIA Tesla T4 GPUs. These accelerators are built on the Turing architecture, each offering 2,560 CUDA cores, 320 Tensor Cores, and 16GB of GDDR6 VRAM, culminating in a combined theoretical capacity of 32GB.[6] The memory bandwidth is rated at 320 GB/s, with a theoretical half-precision (FP16) performance of approximately 65 TFLOPS utilizing Tensor Cores.[6] While robust for standard textual natural language processing, continuous video contextualization inherently triggers severe hardware stress.

The two primary failure modes during such deployments are KV Cache Explosion and Thermal Latency Spikes.[1] During long-context video ingestion, the dynamic Key-Value (KV) cache and compute buffers dominate memory usage, frequently accounting for up to 87% of the total memory footprint at extended context lengths.[1] If a 14B parameter LLM attempts to process uncompressed video frames at FP16 precision, the model weights alone require approximately 32.2GB of VRAM.[7] This allocation leaves an insufficient buffer for the KV cache and spatial-temporal data processing, mathematically guaranteeing an Out-Of-Memory (OOM) fatal exception on the dual T4 setup.

To mitigate this catastrophic failure mode, the architecture mandates the deployment of 4-bit quantized models. Utilizing 4-bit AWQ, the VRAM requirement for a 14B parameter model is reduced to approximately 21.29 GB.[7] This vital compression ensures the model weights fit comfortably across the parallelized T4 GPUs, leaving an 11GB buffer for the KV cache and temporal data processing during the MVBench evaluation.[5] The quantization strategy acts asymmetrically; it is highly effective during the bandwidth-bound decode stage, drastically reducing memory access overhead, though it offers diminishing returns during the compute-bound prefill stage.[1]

Model Scale	**Precision**	**Parameter Count**	**VRAM Requirement (Weights)**	**Kaggle Dual T4 Deployment Viability**
Qwen2.5-VL-3B	FP16	3 Billion	~6.5 GB	Highly Viable (Single T4)
Qwen2.5-VL-7B	FP16	7.61 Billion	16.1 GB	Marginal (Requires 2x T4 and optimization)

Qwen2.5-VL-7B	4-bit AWQ	7.61 Billion	10.65 GB	Highly Viable (Single T4)
Qwen2.5-VL-14B	FP16	14.7 Billion	32.2 GB	Unviable (Extreme OOM Risk)
Qwen2.5-VL-14B	4-bit AWQ	14.7 Billion	21.29 GB	Viable (Requires 2x T4 and vLLM)

Strict Architectural Isolation and Baseline Selection

To eliminate architectural bias and ensure scientific validity within the two-day execution window, both the SLM and the LLM must be selected from the exact same model lineage. Decoupling the visual perception mechanics from the textual reasoning backbone ensures that any observed performance deltas on the MVBench benchmark are mathematically attributed strictly to the parameter scale of the reasoning engine, rather than variations in the underlying visual module or tokenizer.[1]

The Qwen2.5-VL family serves as the optimal framework for this isolation. The architecture utilizes transformers with Multimodal Rotary Position Embedding (M-ROPE), SwiGLU activations, RMSNorm, and Attention QKV biases.[9] The M-ROPE mechanism explicitly decomposes positional embedding into distinct parts to capture one-dimensional textual, two-dimensional visual, and three-dimensional video positional information, enhancing its multimodal processing capabilities without requiring a dedicated three-dimensional video encoder.[10] The baseline SLM selected for this execution is the Qwen2.5-VL-3B, while the LLM counterpart is the Qwen2.5-VL-14B.[1]

To guarantee that both models perceive the exact same visual stimuli, they are equipped with an identical vision encoder. The implementation utilizes a robust, pre-trained Vision Transformer (ViT), specifically the CLIP ViT-L/14 architecture, to extract spatial features and visual patches from the extracted video frames.[1] The weights of this vision encoder are explicitly frozen across all comparative experiments. Targeted research within the design space of visually-conditioned language models demonstrates that fully fine-tuning the visual backbone during the alignment phase can introduce unwanted training variables and degrade performance on tasks requiring fine-grained spatial reasoning.[1] By locking the encoder's parameters, the pipeline guarantees that the base visual representations passed to the 3B model and the 14B model are mathematically identical.

The connector module is responsible for mapping the dense, high-dimensional visual features from the frozen vision encoder into the text embedding space of the language models. To avoid the hidden computational biases associated with complex, query-based routing mechanisms such as Q-Formers, the pipeline initializes a streamlined, two-layer Multi-Layer Perceptron (MLP) projector.[1] Both the SLM and the LLM utilize this exact same MLP architecture to bridge the modalities. This structural uniformity ensures that the translation of visual data into language embeddings is completely standardized, leveling the sensory input before autoregressive textual reasoning is initiated.[1]

Computational Standardization and Inference Budgeting

A critical failure in legacy scaling research is the allowance of massive models to process immense volumes of visual tokens without accounting for the exponential computational cost. To render the experiment scientifically rigorous and definitively prove the core hypothesis, a fixed inference compute budget is enforced. The inference floating-point operations per second (FLOPs) of a Vision-Language Model are dictated by the parameter size of the language model multiplied by the visual token count.[1]

The total theoretical compute cost for every single inference run must be programmatically tracked to establish a compute-optimal frontier. The strict per-example inference compute cost is formulated as a mathematical function of multiple interacting variables. The equation relies on the number of sampled video frames, the parameter count of the vision model, the visual features output by the vision model, the parameter count of the language reasoning backbone, and the final visual tokens per frame passed to the attention layers.[1] By isolating the language model size and treating it as the primary variable against the visual token density, the pipeline can accurately map the trade-offs required for a strict compute budget.

The baseline computational ceiling is established using the 3B SLM processing a high-fidelity token stream. The SLM is permitted to ingest a densely sampled array of video frames with a relatively dense spatial token count per frame, such as one hundred and forty-four tokens per frame across thirty-two frames.[1] The total FLOPs required for the SLM to process this query is calculated and locked in as the absolute Inference Compute Budget for the entirety of the Kaggle execution. Any inference request that structurally exceeds this mathematical budget is actively blocked by the orchestration script.[1]

To ensure absolute fairness, the 14B LLM is mathematically constrained to the exact same Anchor Compute Budget. Because the parameter count of the 14B model is exponentially larger than the 3B model, the visual token count must be aggressively slashed to prevent the equation from exceeding the budget.[1] Recent scaling laws demonstrate that for a fixed compute budget on complex visual reasoning tasks, the optimal strategy dictates deploying the largest possible LLM while applying extreme visual token compression.[1]

This research completely subverts the standard practice of moderate token reduction. It

mathematically proves that for visual reasoning tasks under a fixed inference compute budget, the optimal strategy is to deploy extreme token compression, reducing the visual input to as few as one to four tokens.[1] The LLM's visual input is compressed down to a highly dense representation utilizing algorithms such as One Token Per Image (1TPI) or convolutional TokenPackers.[1] By discarding over ninety-seven percent of the original visual token payload, the architecture dramatically reduces the inference cost.[1] This reallocation of cognitive load ensures that the computational constraints of the dual T4 edge deployment are respected without starving the massive LLM of foundational contextual geometry.

Engineering the Deterministic Data-Ingestion Pipeline

To eliminate the black-box preprocessing biases inherent to default model wrappers, the methodology mandates the construction of a custom, deterministic frame-sampling pipeline. This pipeline guarantees that both the SLM and the LLM receive a mathematically pristine array of visual information, ensuring that task performance is derived from parametric reasoning rather than arbitrary software variations.[1]

Optimized computer vision libraries, specifically OpenCV and FFmpeg, are deployed to extract video frames at a strict, standardized temporal rate.[1] This physical standardization ensures sensory parity across the evaluation suite. All input data is subsequently structured into a Universal Interleaved Formatting template. This format mathematically unifies spatial and temporal elements into one continuous sequence, allowing the model to process the data natively without requiring complex architectural branching or separate modality-specific decoders.[1]

Specialized demarcation tokens are injected directly into the flattened token stream to explicitly signal temporal dependency to the language model. The entire video sequence is enclosed with structural markers, while temporal tokens are inserted between consecutive frames to explicitly enforce chronological order and trajectory.[1] This provides a strict, architecture-independent structure to the flattened array, allowing the model to track object permanence and action transitions natively.

To mitigate the sub-scaling phenomenon—a documented scaling behavior where high data density and redundant information restrict scaling efficiency—the pipeline applies aggressive semantic filtering.[1] When models are prompted with high-density, repetitive data, they encounter a saturation of unique information, causing performance improvements to stall prematurely.[1] The pipeline computes the information density of the raw video dataset by evaluating both intra-cluster concentration and inter-cluster separation. Videos or frames that fall into high-density, redundant clusters are discarded to ensure every sample provides maximum unique informational entropy.[1]

Semantic-based deduplication is subsequently enforced by utilizing the CLS token of a pre-trained language model to extract the global semantics of the video context. A new sequence is only passed into the reasoning engine if its maximum cosine similarity to the

existing contextual pool falls below a strict threshold.[1] This data-centric curation guarantees that the evaluation measures true cognitive capacity rather than the model's ability to memorize repetitive spatial noise.

Dynamic Visual Compression Mechanisms

Standard uniform sampling introduces severe perception bottlenecks by blindly skipping informative visual cues or continuously transmitting redundant static backgrounds. To optimize the data before it reaches the Qwen2.5-VL attention layers, a specialized five-tier dynamic compression strategy is executed. This architecture isolates salient temporal events, aggressively prunes spatial noise, and hierarchically merges features to prevent token overload.[1]

The first tier executes query-agnostic temporal segmentation to parse the unstructured video timeline into discrete, logical events. The pipeline utilizes Continuous Density-Peak Clustering (C-DPCKNN) for this segmentation.[1] Frame-level representations are extracted via the CLIP vision encoder, and local density and distance indices are calculated. Event boundaries are mathematically established at minimum density peaks.[1] Unlike standard uniform sampling which blurs chronological boundaries, this method isolates moments of drastic visual change, resulting in strict, chronologically accurate event parsing that effectively resolves the issue of information redundancy in long videos.[1]

The second tier implements Differential Keyframe Selection (DKS) to identify the highest-value temporal moments within those segmented events. The pipeline calculates the cosine similarity between each frame's visual embedding and the embedding of the user's textual query. A greedy algorithm sorts the frames by their query-relevance score. To prevent the selection of visually identical, redundant frames, a strict temporal distance threshold is enforced. This guarantees that the selected keyframes are both highly relevant to the specific prompt and temporally distinct, avoiding the allocation of compute to redundant spatial clusters.[1]

The third tier introduces temporal dynamic compression to capture rapid, fleeting actions that rigid selection algorithms inherently overlook. A base ratio of frames is sampled uniformly to establish a temporal baseline, and supplementary frames are subsequently selected iteratively.[1] The algorithm targets unchosen candidates exhibiting the lowest cosine similarity relative to the chosen set, indicating the highest semantic variation.[1] This dynamic selection is crucial for tasks within the MVBench evaluation that require fine-grained pose tracking or rapid state changes.

The fourth tier shifts from temporal selection to adaptive spatial compression. Uninformative spatial background noise within each selected frame is eradicated using a sophisticated two-step spatial pruning method.[1] Initially, magnitude-based pruning is applied. The activation magnitude, defined by the L_2 norm of all visual patch tokens in a frame, is computed as a proxy for visual salience.[1] Tokens that fall below a predefined retention ratio are aggressively pruned, effectively deleting static sky, wall patches, or empty backgrounds from the

computation.[1] Subsequently, semantic token merging is executed. The remaining high-saliency tokens are subjected to cosine similarity grouping. Tokens exceeding a high similarity threshold are consolidated into a single representative token via mean pooling.[1] This preserves the semantic identity of macroscopic objects while drastically reducing the overall token sequence length.

The fifth and final tier executes Differential Feature Merging (DFM) for non-keyframes. Instead of discarding non-keyframes entirely—a practice that permanently destroys the continuous background logic and temporal flow of the video—these frames are aggressively compressed down to a single representative token.[1] This is achieved via differentially weighted pooling, where patches receive higher weights if they maintain relevance to the user's query while exhibiting low temporal redundancy compared to the preceding full-resolution keyframe.[1] This hierarchical bottleneck acts as a dynamic filter, allowing the model to digest both dense spatial data and compressed temporal data seamlessly.

Compression Tier	**Methodology**	**Target Dimension**	**Primary Function**
Tier 1	C-DPCKNN Clustering	Temporal	Identifies drastic visual changes to establish rigid chronological event boundaries.
Tier 2	Differential Keyframe Selection	Temporal	Selects prompt-relevant frames while enforcing strict temporal distance thresholds.
Tier 3	Dynamic Supplementary Selection	Temporal	Iteratively captures high-variance frames to track rapid, fine-grained state changes.
Tier 4	Magnitude Pruning &	Spatial	Eradicates low-activation

	Mean Pooling		background noise and groups identical semantic textures.
Tier 5	Differential Feature Merging	Spatial-Temporal	Compresses non-keyframes into single tokens to maintain continuous chronological logic.

Dual T4 Deployment Orchestration and Quantization

The successful execution of this highly engineered pipeline within the Kaggle dual T4 environment demands precise hardware orchestration. Network input/output and orchestration bottlenecks must be entirely mitigated by executing a fully local deployment; relying on external cloud APIs or distributed file systems introduces data transfer latency that will artificially skew the edge-viability metrics.[1]

To deploy the Qwen2.5-VL-14B-Instruct model on hardware limited to 32GB of cumulative VRAM, the model is initialized utilizing 4-bit Activation-aware Weight Quantization (AWQ).[3] AWQ is strategically selected over standard GPTQ or BitsAndBytes because it preserves a fraction of salient weights (based on activation distributions) in higher precision, minimizing the quantization error that typically degrades fine-grained visual perception tasks.[5] Standard loading mechanisms provided by default wrappers are highly inefficient for distributed environments. Therefore, the architecture relies on the vLLM inference engine to manage execution.[5] The vLLM engine provides superior memory management through PagedAttention, which allocates memory in non-contiguous blocks, virtually eliminating KV cache fragmentation.[12] This is a critical necessity when processing the continuous, long token sequences inherent to the MVBench video evaluation.

To distribute the 14B model across both T4 GPUs, tensor parallelism is invoked via the vLLM configuration. The pipeline utilizes the --tensor-parallel-size 2 flag to split the matrix multiplication workloads evenly across the two distinct devices.[13] Because the dual T4 setup relies on standard PCIe Express connections rather than high-bandwidth NVLink for Peer-to-Peer (P2P) memory sharing, the orchestration script must minimize cross-GPU communication. Tensor parallelism handles the weight distribution efficiently, but the input prompt and sequence length must be mathematically constrained to prevent bandwidth bottlenecks.[13] Thus, the --max-model-len 4096 flag is enforced to cap the autoregressive

generation loop and prevent PCIe saturation during the decode stage.[13]

Furthermore, to prevent the vLLM engine from preemptively reserving one hundred percent of the VRAM (which would leave absolutely no memory for the data-ingestion pipeline's localized FFmpeg extractions and clustering algorithms), the --gpu_memory_utilization 0.95 flag is explicitly deployed.[13] This orchestration guarantees that the Python runtime environment possesses sufficient overhead to execute the complex Python logic required by the five-tier compression algorithm before passing the final tensors to the CUDA devices.

Performance tracking on the edge hardware is separated into two distinct stages to accurately profile the latency bottlenecks. The prefill stage ingests the text prompt alongside the massive sequence of compressed video tokens. Because it processes tokens in parallel, it is highly compute-bound and relies heavily on matrix-vector multiplications within the Feed-Forward Neural Network and Attention layers.[1] The efficiency of the data pipeline is proven here by logging the Time-to-First-Token (TTFT).[1] Once the temporal context is absorbed, the model transitions to the decode stage, generating the textual answer sequentially. This stage is strictly memory-bandwidth-bound. The 4-bit AWQ quantization is vastly more effective here, reducing the memory access overhead and slashing decode latency by up to seventy-five percent.[1] Throughput is measured and logged strictly as Tokens Per Second (TPS) to ensure the system can maintain a stable generation speed without triggering thermal throttling.[1]

Evaluation via the MVBench Benchmark

Due to the strict two-day execution timeframe dictated by the project parameters, the comprehensive evaluation suite is restricted solely to MVBench.[1] MVBench is uniquely suited for assessing deep temporal reasoning, successfully circumventing the spatial memorization biases of traditional static benchmarks that fail to measure motion or causality.[2]

The MVBench framework comprises four thousand rigorous multiple-choice Question-Answer pairs distributed across twenty distinct task categories.[16] The benchmark employs a novel static-to-dynamic conversion methodology, transforming static image tasks into dynamic evolutionary evaluations. This forces the model to utilize a broad spectrum of temporal skills, systematically testing capabilities from low-level perception to high-level cognition.[2]

The evaluation framework mandates that the model processes continuous visual changes to answer questions regarding Action Localization, Scene Transition, Object Interaction, and Fine-grained Pose.[19] For example, static frames cannot reliably distinguish between high kinematic similarity sports sequences like butterfly versus freestyle swimming strokes; the model must evaluate the temporal pattern analysis.[17] By utilizing public video annotations automatically converted into multiple-choice formats, MVBench guarantees evaluation fairness with ground-truth data, avoiding the biased, subjective scoring inherent to LLMs grading open-ended outputs.[2]

MVBench Task Category	Cognitive Requirement	Temporal Skill Tested
Action Sequence	Sequential Logic	Understanding the chronological order of specific actions.
Action Count	Temporal Perception	Calculating the exact frequency of a repeated action over time.
Moving Attribute	Dynamic Tracking	Identifying the appearance of an object exclusively while it is in motion.
State Change	Causal Reasoning	Determining whether the physical state of an object alters throughout the video.
Fine-grained Pose	Kinematic Analysis	Identifying accurate physical postures across continuous frames.

A critical operational challenge for this deployment is the baseline inference runtime. Comprehensive execution of MVBench across unoptimized models can exceed six to twelve hours of continuous GPU processing, primarily due to the computational burden of initial dense frame processing.[20] Kaggle enforces a strict twelve-hour session timeout limit for interactive kernels. Therefore, the implementation of the five-tier dynamic compression strategy and the strict inference FLOP standardization (restricting the 14B model to one to four tokens per frame) is not merely a theoretical exercise; it is an absolute operational necessity. Without these data-centric optimizations, the memory overhead and prolonged prefill latencies would guarantee the kernel terminates before the benchmark execution completes.[1]

The benchmark is evaluated utilizing the exact-match criterion for the multiple-choice tasks.

The evaluation script parses the generated text, utilizing regular expressions to search for the specific option identifiers corresponding to the ground-truth video annotations. This ensures rapid evaluation and minimizes the token generation length required by the model, further accelerating the decode stage.[2]

The resulting metrics logged in the performance matrix will detail task accuracy across the twenty temporal tasks, prefill latency (TTFT) required to process the video input array, decode throughput (TPS) representing the generation speed of the output tokens, and the peak VRAM footprint.[1] This matrix provides a comprehensive view of the model's intelligence against its physical operational cost.

Conclusions and Pareto-Optimal Projections

By executing this multimodal contextualization pipeline, the project synthesizes both cognitive intelligence and physical hardware metrics into a definitive mathematical analysis of edge-device viability. The core objective is to map the compute-optimal frontier, plotting the average downstream task error on MVBench against the actual inference compute costs, measured by TTFT and VRAM utilization.[1]

The theoretical projections indicate that the data-engineered Qwen2.5-VL-3B SLM—processing highly pristine, semantically filtered, and densely formatted visual tokens—will achieve cognitive reasoning scores commensurate with the Qwen2.5-VL-14B LLM. Because the 14B LLM is forced by the FLOP standardization equation to aggressively compress its visual inputs to a mere fraction of the original frames, it suffers an inherent reduction in fine-grained spatial and temporal perception. The SLM, operating unburdened by severe token compression due to its lightweight parameter count, can leverage the full spectrum of the five-tier data ingestion pipeline to capture nuanced scene transitions, precise action counts, and subtle moving attributes.

Simultaneously, the SLM will process the input sequence with significantly lower prefill latency and a substantially smaller KV cache footprint. If the empirical data confirms that the 3B model matches or exceeds the 14B model's accuracy on the temporal reasoning tasks within MVBench, while successfully avoiding the thermal throttling and OOM risks associated with the dual T4 memory ceiling, the core hypothesis is validated.

This validation establishes that intelligent data curation, extreme visual token compression, and targeted SLM deployment constitute a Pareto-optimal approach for multimodal artificial intelligence. It definitively proves that optimizing the input data pipeline is a mathematically superior and hardware-efficient alternative to the computationally unsustainable paradigm of blind parameter scaling, opening new pathways for advanced video contextualization in constrained edge environments.

Works cited

1. All Research Papers Segment-Wise Synthesis (Segment 1).pdf

2. MVBench: A Comprehensive Multi-modal Video Understanding Benchmark - arXiv, accessed on April 16, 2026, https://arxiv.org/abs/2311.17005
3. Qwen/Qwen2.5-VL-72B-Instruct-AWQ - Hugging Face, accessed on April 16, 2026, https://huggingface.co/Qwen/Qwen2.5-VL-72B-Instruct-AWQ
4. Kaggle upgraded their free tier to T4 with 29GB and 4 CPU cores : r/LocalLLaMA - Reddit, accessed on April 16, 2026, https://www.reddit.com/r/LocalLLaMA/comments/17bhwtj/kaggle_upgraded_their_free_tier_to_t4_with_29gb/
5. Speeding up vllm inference for Qwen2.5-VL - General, accessed on April 16, 2026, https://discuss.vllm.ai/t/speeding-up-vllm-inference-for-qwen2-5-vl/615
6. Multi-GPU Training in Kaggle: Harnessing 2× T4 for Faster Model Training(TensorFlow with MNIST Dataset) | by Harisudhan.S | Medium, accessed on April 16, 2026, https://medium.com/@speaktoharisudhan/multi-gpu-training-in-kaggle-harnessing-2-t4-for-faster-model-training-tensorflow-f091c22db99e
7. GPU System Requirements Guide for Qwen LLM Models (All Variants), accessed on April 16, 2026, https://apxml.com/posts/gpu-system-requirements-qwen-models
8. Deep Learning Performance on T4 GPUs with MLPerf Benchmarks | Dell US, accessed on April 16, 2026, https://www.dell.com/support/kbdoc/en-us/000132094/deep-learning-performance-on-t4-gpus-with-mlperf-benchmarks
9. unsloth/Qwen2.5-14B-Instruct - Hugging Face, accessed on April 16, 2026, https://huggingface.co/unsloth/Qwen2.5-14B-Instruct
10. qwen2-vl-7b - Kaggle, accessed on April 16, 2026, https://www.kaggle.com/datasets/humbleyll/qwen2-vl-7b
11. Qwen 2.5 14B Instruct AWQ vLLM - Kaggle, accessed on April 16, 2026, https://www.kaggle.com/code/cemalemrealbayrak/qwen-2-5-14b-instruct-awq-vllm
12. Qwen2.5 VL 72B VRAM Needs: Why Vision Models Demand More - Novita, accessed on April 16, 2026, https://blogs.novita.ai/qwen2-5-vl-72b-vram/
13. is there any quantized models for Qwen2.5-VL · Issue #719 · QwenLM/Qwen3-VL - GitHub, accessed on April 16, 2026, https://github.com/QwenLM/Qwen3-VL/issues/719
14. Qwen/Qwen2.5-14B-Instruct-1M - Hugging Face, accessed on April 16, 2026, https://huggingface.co/Qwen/Qwen2.5-14B-Instruct-1M
15. qwen2.5-vl-7b - Kaggle, accessed on April 16, 2026, https://www.kaggle.com/code/mengaidev/qwen2-5-vl-7b
16. MT-Video-Bench: A Holistic Video Understanding Benchmark for Evaluating Multimodal LLMs in Multi-Turn Dialogues - arXiv, accessed on April 16, 2026, https://arxiv.org/html/2510.17722v2
17. MVPBench: A Multi-Video Perception Evaluation Benchmark for Multi-Modal Video Understanding - arXiv, accessed on April 16, 2026, https://arxiv.org/html/2603.22756v1
18. MVBench: Comprehensive Video Benchmark | PDF | Perception | Cognitive

Science - Scribd, accessed on April 16, 2026, https://www.scribd.com/document/832483378/Li-MVBench-a-Comprehensive-Multi-modal-Video-Understanding-Benchmark-CVPR-2024-Paper
19. MVBench: A Comprehensive Multi-modal Video Understanding Benchmark - CVF Open Access, accessed on April 16, 2026, https://openaccess.thecvf.com/content/CVPR2024/papers/Li_MVBench_A_Comprehensive_Multi-modal_Video_Understanding_Benchmark_CVPR_2024_paper.pdf
20. Frame Sampling Strategies Matter: A Benchmark for small vision language models - arXiv, accessed on April 16, 2026, https://arxiv.org/html/2509.14769v2